TRƯỜNG ĐẠI HỌC KHOA HỌC

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỀ TÀI

HỆ THỐNG MỞ KHÓA CỬA BẰNG CẢM BIẾN CHUYỂN ĐỘNG

Tên lớp học phần:Phát triển ứng dụng IoT

Mã học phần: 2025-2026.2.TIN4024.006

Giảng viên hướng dẫn: TS.Lê Hữu Bình

Sinh viên thực hiện : Nhóm 6

22T1020193 - Đỗ Trọng Hoàng Lân

22T1020659 - Đặng Văn Long

Huế,tháng 04 năm 2026

MỤC LỤC

I. ĐẶT VẤN ĐỀ **1**
II. NỘI DUNG CỦA ĐỀ TÀI **2**
1.Các nội dung thực hiện **2**
1.1. Thu thập và Xử lý dữ liệu (tại ESP32) 2
1.2 Truyền thông và Kết nối (MQTT) 2
1.3 Giám sát và Điều khiển (Node-Red) 3
2.Thiết kế mạch IoT **3**
2.1. Sơ đồ logic của hệ thống (EasyEDA) 3
2.2 Giải thích chức năng cơ bản của các linh kiện, thiết bị sử dụng trong mạch 4
2.3 Nguyên lý hoạt động 5
2.4 Mạch vật lý (Mô phỏng) 8
3.Cài đặt hệ thống **8**
3.1 Công cụ cài đặt 8
3.2 Quy trình cài đặt 9
4. Kết quả thực hiện **11**
4.1 Khởi động 11
4.2 Xử lý theo cảm biến PIR 11
4.3 Kiểm tra hệ thống (Hệ thống tự động) 15
5. Kết luận hướng phát triển **18**
5.1. Các công việc đã thực hiện được 18
5.2. Các công việc chưa thực hiện được 19
5.3. Hướng phát triển của đề tài 19
PHỤ LỤC **21**

I. ĐẶT VẤN ĐỀ

Việc triển khai Hệ thống mở khóa cửa bằng cảm biến chuyển động không chỉ là một bước tiến về công nghệ mà còn giải quyết triệt để nhiều vấn đề trong đời sống hiện đại ngày nay.

Hệ thống mở cửa tự động sử dụng cảm biến chuyển động (thường là cảm biến hồng ngoại PIR hoặc sóng Microwave) hoạt động dựa trên nguyên lý nhận biết sự thay đổi của môi trường khi có thực thể di chuyển vào vùng quét. Khi phát hiện tín hiệu, bộ điều khiển trung tâm sẽ kích hoạt mô-tơ điện để mở cánh cửa mà không cần bất kỳ tác động lực nào từ con người..

Thực tế cho thấy việc sử dụng tay nắm cửa vật lý bộ lộ nhiều hạn chế trong đời sống như: mang vác vận chuyển vật nặng, vật cồng kềnh, khó khăn đối với người khuyết `tật.Đặc biệt trong bối cảnh đại dịch COVID-19 vừa rồi việc hạn chế tiếp xúc (không chạm) bề lên các bề mặt trong đó có cửa ra vào càng trở nên cấp thiết hơn bao giờ hết.

Cùng với đó Hệ thống mở khóa cửa bằng cảm biến chuyển động còn giúp cho việc vận hành môi trường công nghiệp,các trung tâm thương mại,...được diễn ra thuận lợi khi luân chuyển hàng hóa,luồng người ra vào giảm tình trạng ùn tắc ở các lối ra vào.Hệ thống này còn tiết kiệm năng lượng cho các tòa nhà hiện đại,các hội trường lớn sử dụng điều hòa khi việc đóng cửa tự động giảm sự thất thoát và tiết kiệm điện. Mặc dù có tính ứng dụng cao, hệ thống này vẫn đối mặt với bài toán về bảo mật (tránh việc cửa mở ngoài ý muốn do vật nuôi hoặc vật thể bay ngang) và chi phí lắp đặt ban đầu. Tuy nhiên,với sự phát triển của AI và các thuật toán lọc nhiễu, hệ thống cảm biến ngày nay đã trở nên chính xác hơn, có khả năng phân biệt giữa chuyển động của con người và các yếu tố môi trường.

II. NỘI DUNG CỦA ĐỀ TÀI

1.Các nội dung thực hiện

Với nguyên lý hoạt động cơ bản sau: Phát hiện có người(mở cửa),Cảnh báo khi cửa mở sai cách (quá thời gian mở),Đếm số lần đóng mở để bảo,Thông báo cho email nếu xảy ra lỗi, hiển thị UI node-red. Nhóm đã thực hiện mô phỏng trên thiết bị ESP32 thông qua giao thức kết nối MQTT và công cụ Node-RED.

1.1. Thu thập và Xử lý dữ liệu (tại ESP32)

- **Phát hiện trạng thái:** Cảm biến PIR quét chuyển động; khi có người, chân GPIO 13 nhận tín hiệu mức cao (HIGH) xác định trạng thái "Cửa mở".
- **Hiển thị thông báo mở cửa :** LED (GPIO 2) sáng và Buzzer (GPIO 12/4) phát âm thanh thông báo ngắn.
- **Đo lường thời gian:** Sử dụng hàm `millis()` để tính toán thời gian thực từ lúc mở đến lúc đóng.Đếm tổng thời gian mở cửa để thống kê tương quan lưu lượng khách, hết ngày thì reset lại.
- **Cảnh báo an toàn:** Nếu thời gian mở vượt quá ngưỡng 10 phút, Buzzer sẽ hú liên tục để báo động.

1.2 Truyền thông và Kết nối (MQTT)

- **Gửi dữ liệu:** ESP32 đóng gói các thông tin (Thời gian mở, Cảnh báo lỗi) thành các bản tin và gửi lên Broker (HiveMQ) thông qua mạng WiFi.
- **Phân luồng Topic:** *topic_data = "iot/door/data"* để nhận dữ liệu các thông báo như (thời gian mở,cảnh báo lỗi), *topic_control = "iot/door/control"* điều khiển trạng thái hoạt động của ESP32

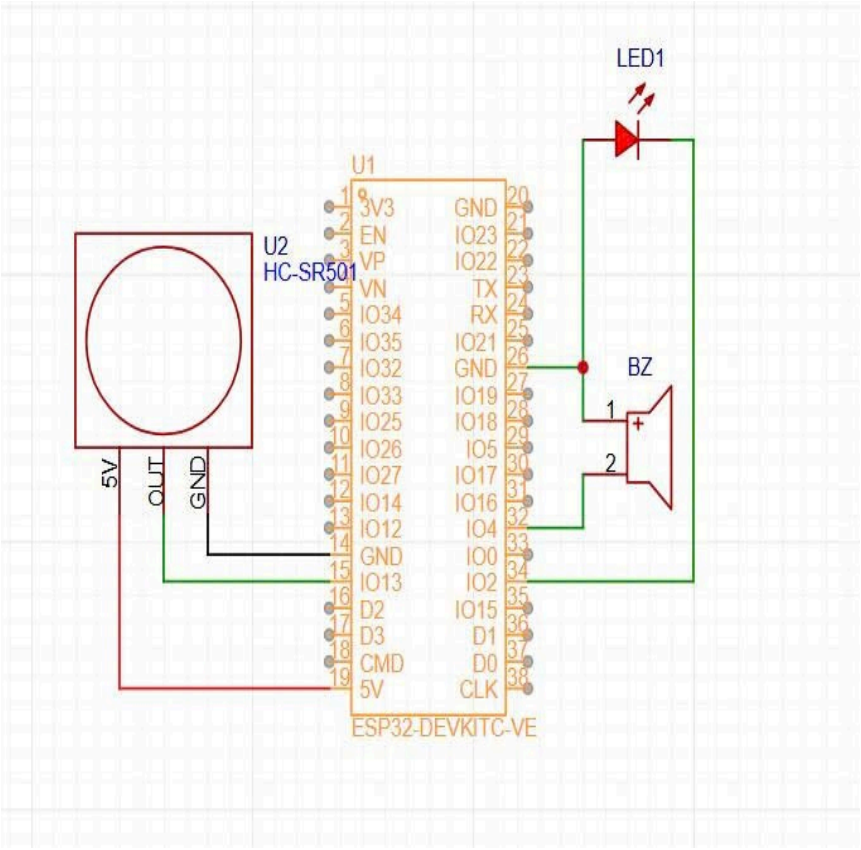
1.3 Giám sát và Điều khiển (Node-Red)

Xử lý dữ liệu: Node-Red nhận dữ liệu từ MQTT để thực hiện

- **Hiển thị UI:** Trình diễn trực quan trạng thái cửa, biểu đồ lưu lượng và danh sách cảnh báo lên Dashboard cho người quản lý.
- **Thông báo từ xa:** Tự động kích hoạt gửi Email cảnh báo khi nhận được tín hiệu "warning" hoặc khi số lần đóng mở đạt ngưỡng bảo trì.

2.Thiết kế mạch IoT

2.1. Sơ đồ logic của hệ thống (EasyEDA)



2.2 Giải thích chức năng cơ bản của các linh kiện, thiết bị sử dụng trong mạch

- **Mạch vi điều khiển ESP32 (ESP32 DevKit C)**

→ ***Chức năng cơ bản:*** *Là một bảng mạch vi điều khiển mạnh mẽ, có tích hợp sẵn module WiFi và Bluetooth. Nó hoạt động như "bộ não" của toàn bộ hệ thống.*

- **Cảm biến chuyển động PIR (HC-SR501).**

→ ***Chức năng cơ bản:*** *Nhận diện sự thay đổi của tia hồng ngoại phát ra từ cơ thể người (hoặc động vật lớn) khi họ di chuyển trong vùng quét của cảm biến.*

- **Đèn LED (Mô phỏng trạng thái Cửa)**

→ ***Chức năng cơ bản:*** *Phát sáng khi có dòng điện chạy qua.*

- **Còi chip (Buzzer)**

→ **Chức năng cơ bản:** Phát ra âm thanh (tiếng bíp hoặc tiếng hú) khi được cấp điện.

2.3 Nguyên lý hoạt động

BƯỚC 1: Khởi động (Setup)

- Hệ thống khởi tạo các chân ngoại vi (PIR-13, LED-2, Buzzer-4).
- Kết nối WiFi `Wokwi-GUEST` và máy chủ MQTT HiveMQ (Port 8883).
- Đăng ký hàm `callback` để sẵn sàng nhận lệnh điều khiển từ topic `iot/door/control`.
- Ghi lại mốc thời gian bắt đầu hoạt động để tính chu kỳ 24h.

BƯỚC 2: Kiểm tra hệ thống (Hệ thống tự động)

1. **Kiểm tra chu kỳ thời gian 24h (Tự động Reset dữ liệu)**
   - Hệ thống sử dụng biến `lastResetMillis` để theo dõi thời gian hoạt động.
     + **Hành động:** Nếu thời gian trôi qua vượt quá 86.400.000 ms (tương đương 24 giờ), ESP32 sẽ tự động:
     + Reset số lần đóng/mở (`doorCycleCount`) về **0**.
     + Reset tổng thời gian mở trong ngày (`totalDailyOpenTime`) về **0**.

+ Gửi bản tin JSON lên Node-RED với sự kiện `"daily_reset"` để đồng bộ hóa biểu đồ trên Dashboard.

2. **Kiểm tra chế độ điều khiển:**
   - Nếu đang ở chế độ THỦ CÔNG (Manual Mode - UNDER_MAINTENANCE):
     + **Ưu tiên:** Đây là chế độ ưu tiên cao nhất, dùng cho mục đích sửa chữa hoặc dọn dẹp.
     + **Hành động: * Ép bật LED:** Giữ đèn sáng liên tục để báo hiệu và hỗ trợ ánh sáng.
     + **Vô hiệu hóa Buzzer:** Tắt mọi tiếng còi báo động để tránh gây phiền nhiễu khi đang có người sửa chữa.
     + **Ngừng đọc PIR:** ESP32 bỏ qua hoàn toàn các tín hiệu từ cảm biến chuyển động. Không đếm thêm lượt, không tính thêm thời gian.
     + **Trạng thái gửi về Node-RED:** Luôn đính kèm nhãn `maintenance: "UNDER_MAINTENANCE"` để Dashboard hiển thị màu vàng cảnh báo đang bảo trì.
   - Nếu ở chế độ TỰ ĐỘNG: Tiếp tục sang Bước 3 để đọc cảm biến.

BƯỚC 3: Rẽ nhánh xử lý theo Cảm biến PIR

Trường hợp A: Cửa bắt đầu mở (PIR: LOW -> HIGH)

- Đánh dấu trạng thái `isDoorOpen = true`.
- Lưu mốc thời gian bắt đầu mở (`openStartTime`).
- Bật đèn LED và phát tiếng bíp ngắn (1500Hz).

Trường hợp B: Người vẫn đứng đó/Cửa tiếp tục mở *(PIR giữ ở mức CAO)*

- Tính thời gian đã mở. Nếu vượt quá `MAX_OPEN_TIME` *(trong code đang để là 10 giây)*
- Hú còi liên tục (2000Hz).
- Đóng gói dữ liệu JSON gồm: tổng thời gian trong ngày, số lần đóng mở, trạng thái bảo trì *(REQUIRED nếu mở > 5 lần)* và gửi lên MQTT topic `iot/door/data.`

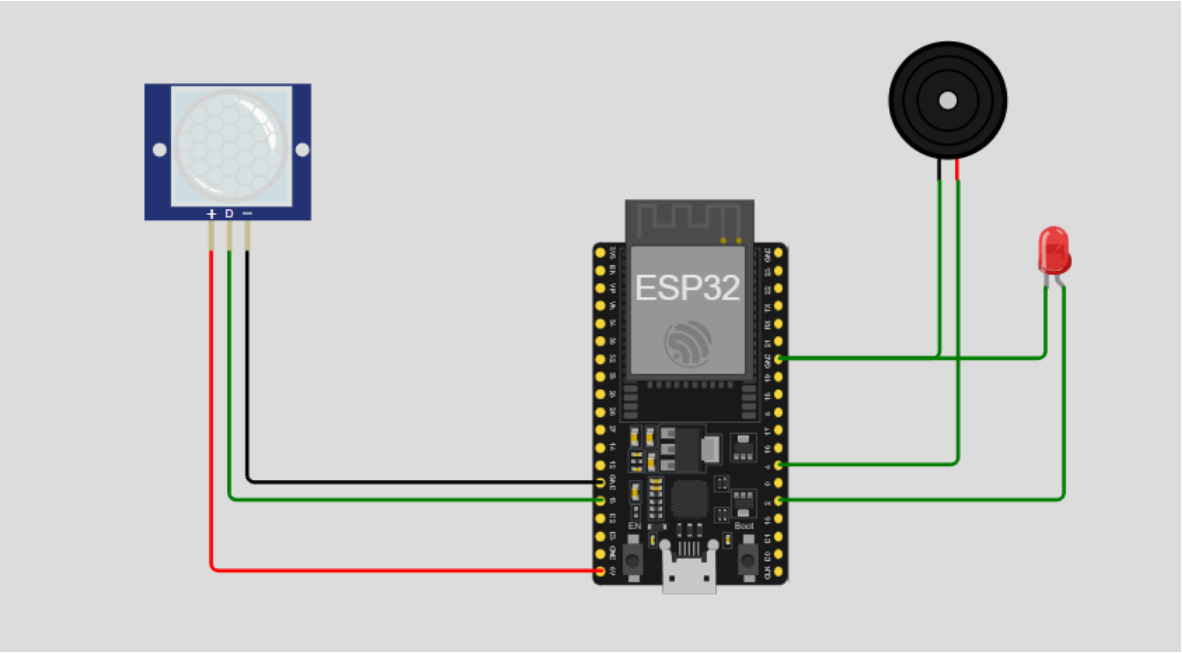
Trường hợp C: Người rời đi/Cửa đóng *(PIR: HIGH -> LOW)*

- Tắt LED, dừng hú còi báo động.
- Phát tiếng bíp xác nhận đóng cửa (800Hz).
- **Cập nhật thống kê:** Cộng dồn thời gian mở vào `totalDailyOpenTime` và tăng số lần đóng mở `doorCycleCount` lên 1.
- **Gửi báo cáo:** Gửi dữ liệu JSON lên Node-Red để cập nhật Dashboard.

BƯỚC 4: Tạm dừng & Lặp lại

- Hệ thống tạm nghỉ 50ms (để chống nhiễu tín hiệu).
- Quay trở lại **BƯỚC 2.**

2.4 Mạch vật lý (Mô phỏng)



3.Cài đặt hệ thống

3.1 Công cụ cài đặt

- Phần cứng (Hardware):Esp32 dev kit, LED, PIR (HC-SR501), Buzzer
- Phần mềm & Thư viện (Software & Libraries):
  + Môi trường lập trình (IDE): Arduino IDE, Visual Studio Code (kèm PlatformIO), mô phỏng trực tuyến Wokwi, ngôn ngữ: C/C++, thư viện lập trình ESP32: *WiFi.h, WiFiClientSecure.h, PubSubClient.h: ArduinoJson.h*
- Nền tảng Đám mây & Quản lý:
  + Broker MQTT, Node-RED

3.2 Quy trình cài đặt

- Mô phỏng hệ thống bằng mạch Esp32, với LED giả lập cửa đóng mở cùng còi cảnh báo trên cài đặt trên Wokwi.
- Thiết lập môi trường giao thức MQTT bằng Website HiveMQ với kênh nhận dữ liệu "iot/door/data" từ Esp32 lên Cloud ,và điều khiển xuống Esp32 từ kênh "iot/door/control".
- Cấu hình Node-RED
  + Nhận và Hiển thị dữ liệu (Subscribing)

    MQTT In Node: Cấu hình Server HiveMQ và Topic là iot/door/data

    JSON Node: Chuyển đổi chuỗi nhận được thành đối tượng để truy xuất dữ liệu.

    Dashboard Chart/Gauge:

    Hiển thị **Tổng thời gian mở cửa** trong ngày.

    Hiển thị **Số lần đóng mở** (Cycle Count).

    Text Node: Hiển thị trạng thái bảo trì,cảnh báo.

  + Điều khiển từ xa (Publishing)

    Dashboard Switch: Tạo một công tắc trên giao diện với hai nhãn "Thủ công" và "Tự động".

    Change Node:

    Nếu Switch ON -> Payload = `MANUAL_ON`.

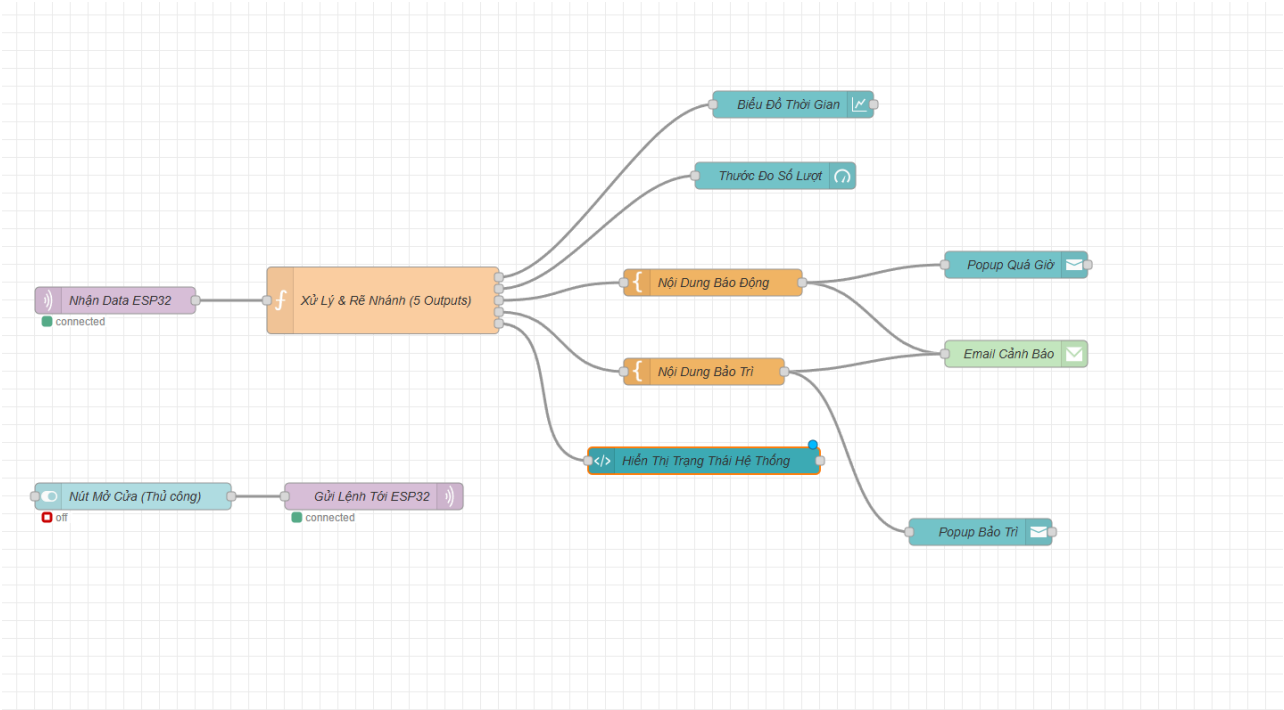
Nếu Switch OFF -> Payload = `AUTO`.

MQTT Out Node: Gửi lệnh xuống Topic `iot/door/control`.

+ Gửi thông báo Email (Logic bổ sung)

Function Node: Kiểm tra nếu gói JSON nhận được có trường `event: "warning"` (mở quá thời gian).

Email Node: Tự động gửi thư cảnh báo đến địa chỉ email của bạn ngay lập tức khi phát hiện lỗi.



4. Kết quả thực hiện

4.1 Khởi động

- Kết nối Wifi Wokwi và MQTT HiveMQ

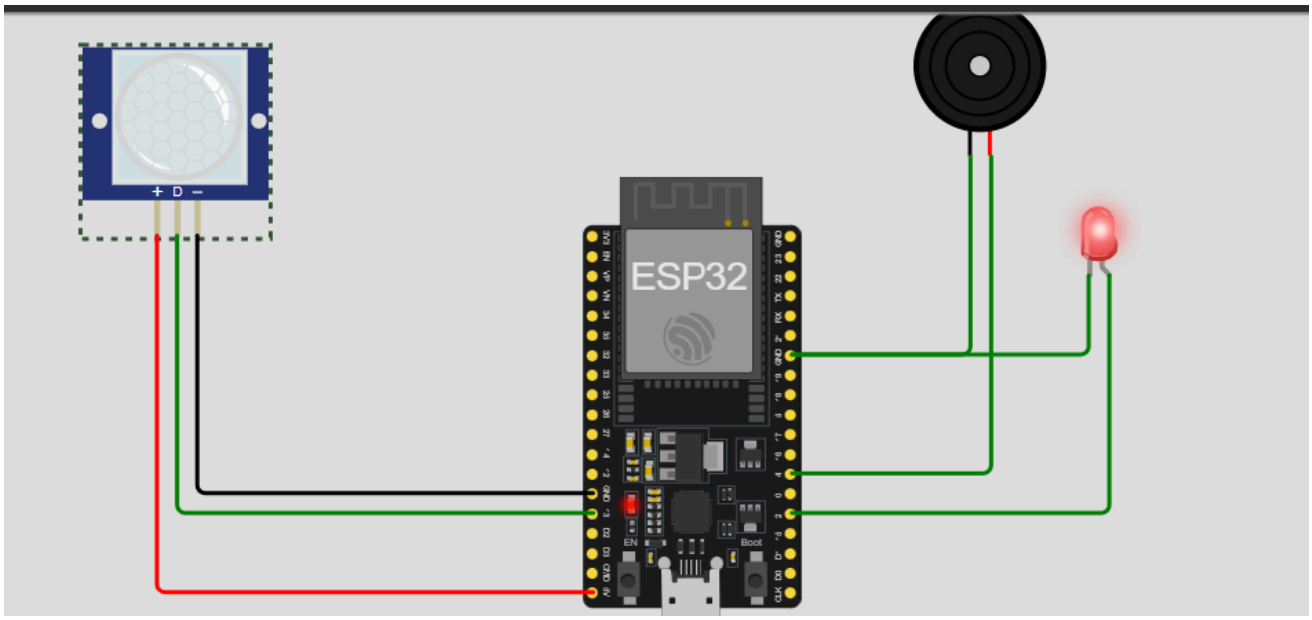
```
Connecting to WiFi.....
                    WiFi Connected!
Attempting MQTT connection...Connected!
```

4.2 Xử lý theo cảm biến PIR

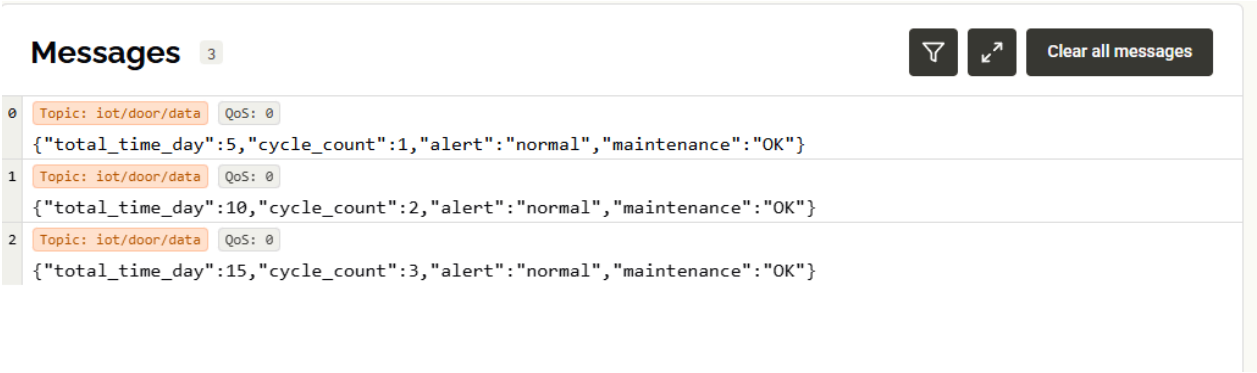
TH 1: Cửa mở bình thường

- Cửa bắt đầu mở (PIR: LOW -> HIGH)

```
Data sent: {"total_time_day":5,"cycle_count":1,"alert":"normal","maintenance":"OK"}
Data sent: {"total_time_day":10,"cycle_count":2,"alert":"normal","maintenance":"OK"}
Data sent: {"total_time_day":15,"cycle_count":3,"alert":"normal","maintenance":"OK"}
```



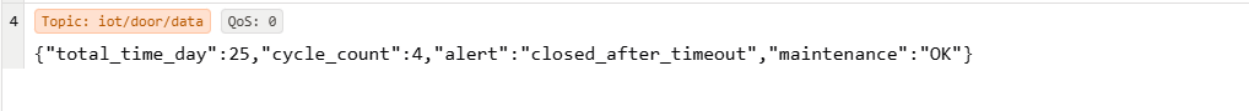
- Gửi lên cloud HieMQ từ topic `iot/door/data`

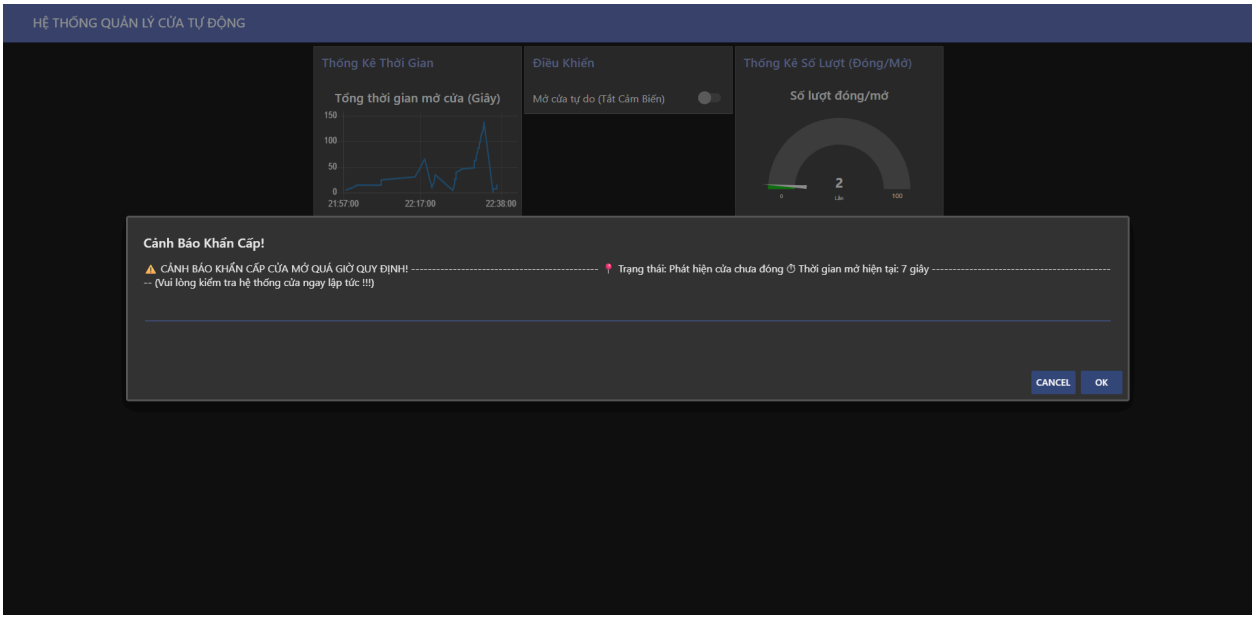


TH 2: Cửa mở sai cách

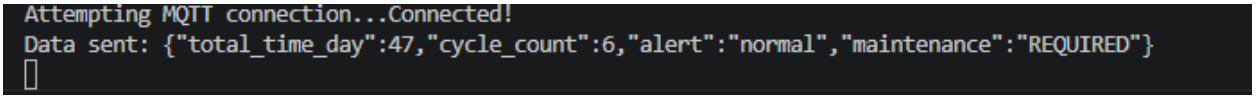
- PIR giữ ở mức CAO, nếu vượt quá thời gian mở cửa thì cảnh báo

```
Data sent: {"event":"warning","total_time_day":15,"cycle_count":3,"alert":"Door left open > 10m","maintenance":"OK"}
```





- Cảnh báo bảo trì nếu vượt ngưỡng

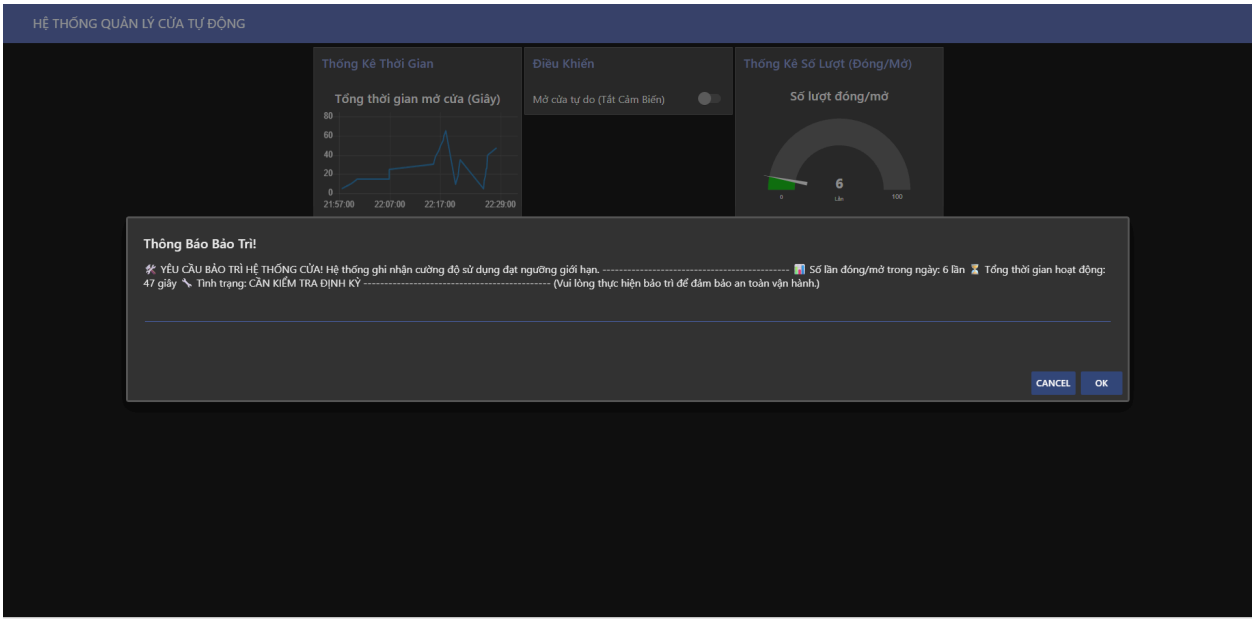


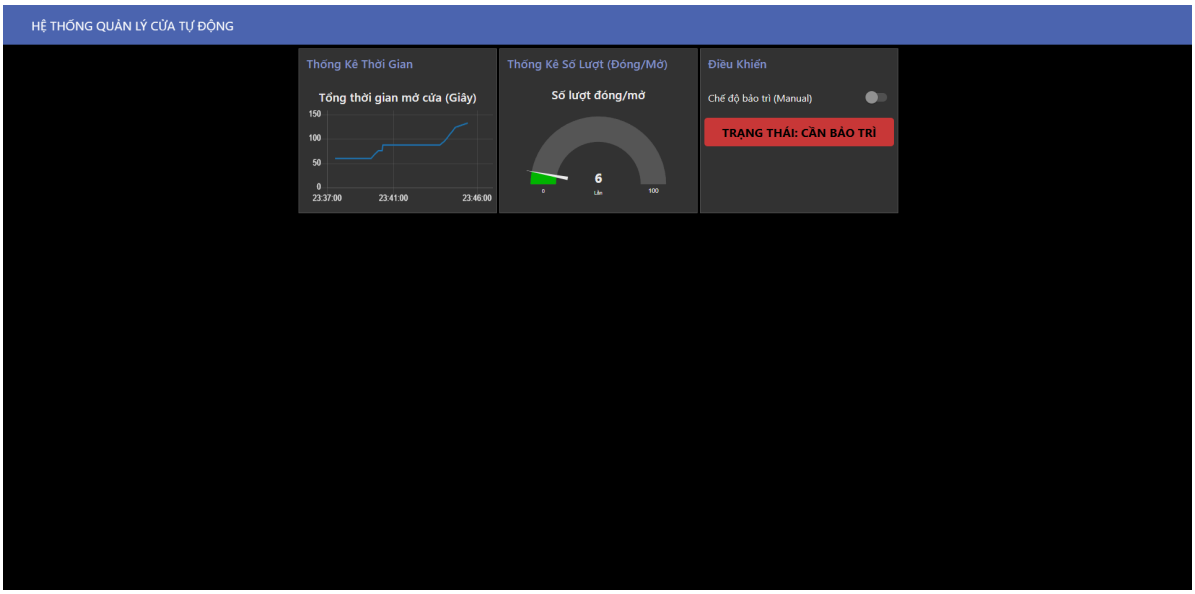
Topic: iot/door/data QoS: 0
{"total_time_day":38,"cycle_count":6,"alert":"normal","maintenance":"REQUIRED"}
Topic: iot/door/data QoS: 0
{"total_time_day":44,"cycle_count":7,"alert":"normal","maintenance":"REQUIRED"}
Topic: iot/door/data QoS: 0
{"total_time_day":50,"cycle_count":8,"alert":"normal","maintenance":"REQUIRED"}
Topic: iot/door/data QoS: 0
{"total_time_day":55,"cycle_count":9,"alert":"normal","maintenance":"REQUIRED"}
Topic: iot/door/data QoS: 0

iot/door/data
External
Hộp thư đến ×
danglongthptlethuy@gmail.com
đến tôi
22:28 (0 phút trước)
🛠️ YÊU CẦU BẢO TRÌ HỆ THỐNG CỬA!
Hệ thống ghi nhận cường độ sử dụng đạt ngưỡng giới hạn.

📊 Số lần đóng/mở trong ngày: 6 lần
⏳ Tổng thời gian hoạt động: 47 giây
🔧 Tình trạng: CẦN KIỂM TRA ĐỊNH KỲ

(Vui lòng thực hiện bảo trì để đảm bảo an toàn vận hành.)

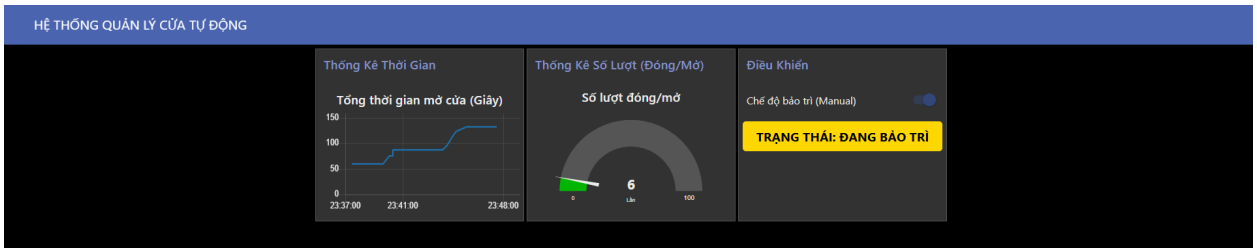

HỆ THỐNG QUẢN LÝ CỬA TỰ ĐỘNG
Thống Kê Thời Gian
Tổng thời gian mở cửa (Giây)
80
60
40
20
0
21:57:00
22:07:00
22:17:00
22:29:00
Điều Khiển
Mở cửa tự do (Tắt Cảm Biến)
Thống Kê Số Lượt (Đóng/Mở)
Số lượt đóng/mở
6
Lần
0
100
Thông Báo Bảo Trì!
🛠️ YÊU CẦU BẢO TRÌ HỆ THỐNG CỬA! Hệ thống ghi nhận cường độ sử dụng đạt ngưỡng giới hạn. ---------- 📊 Số lần đóng/mở trong ngày: 6 lần ⏳ Tổng thời gian hoạt động: 47 giây 🔧 Tình trạng: CẦN KIỂM TRA ĐỊNH KỲ ---------- (Vui lòng thực hiện bảo trì để đảm bảo an toàn vận hành.)
CANCEL
OK

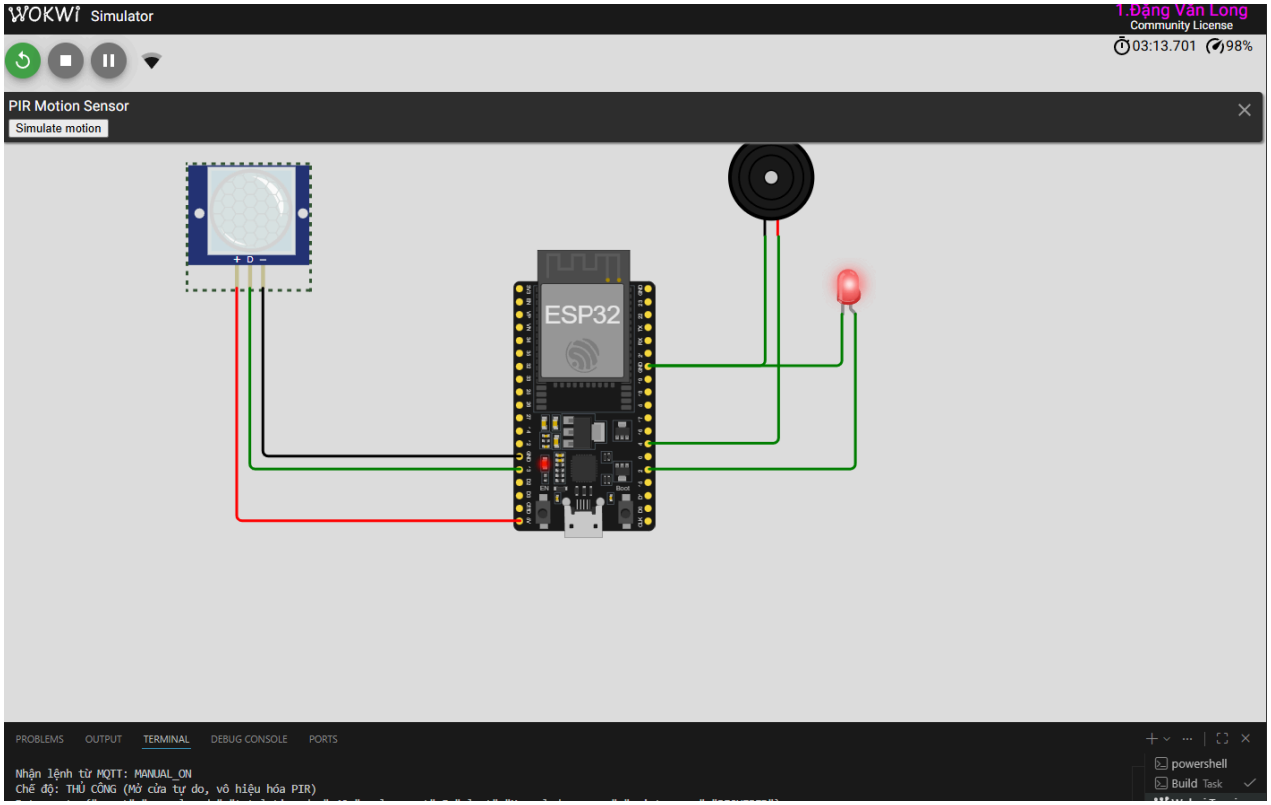


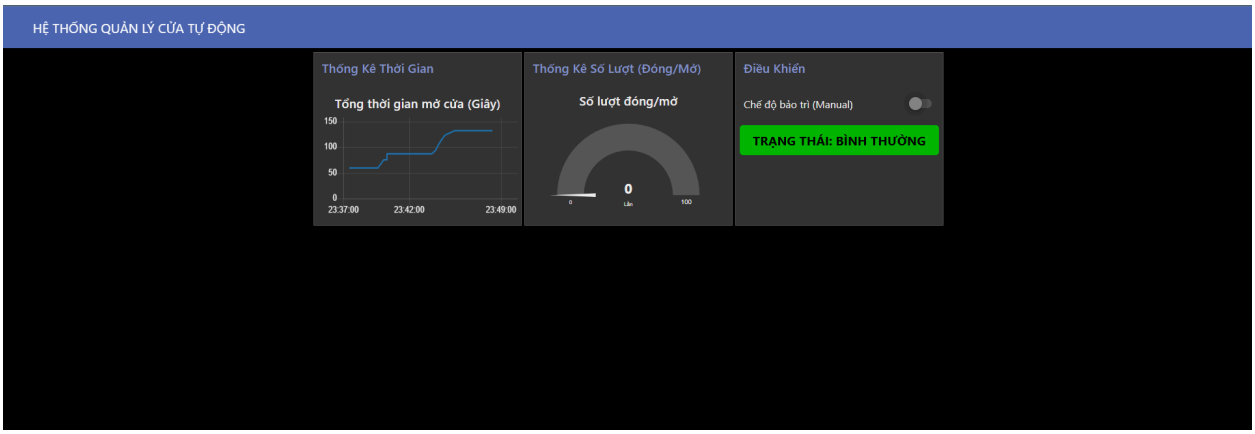
4.3 Kiểm tra hệ thống (Hệ thống tự động)

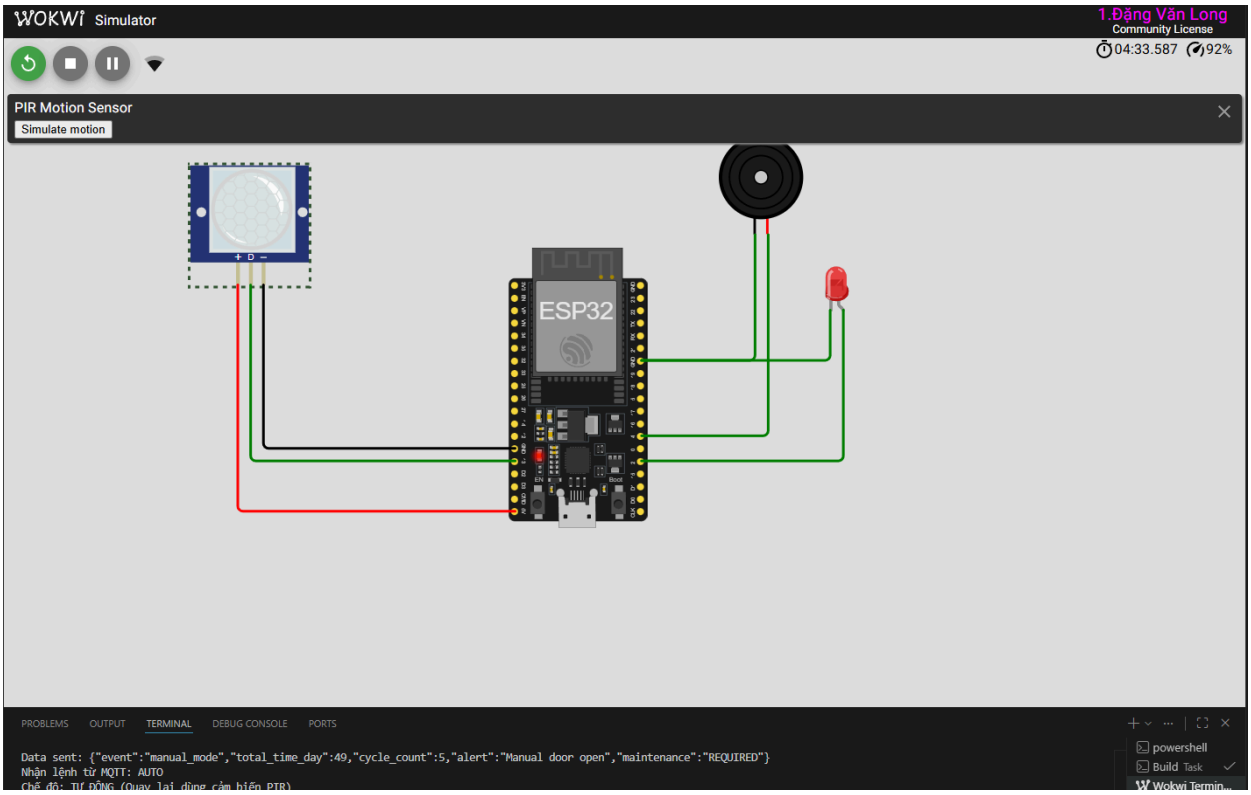
- **Kiểm tra chế độ điều khiển:**

  - Nếu đang ở chế độ THỦ CÔNG (Manual Mode): Ép bật LED, tắt còi, và ngừng đọc cảm biến PIR. Hệ thống đứng yên chờ lệnh mới.
  - Nếu ở chế độ TỰ ĐỘNG: Tiếp tục để đọc cảm biến.


HỆ THỐNG QUẢN LÝ CỬA TỰ ĐỘNG
Thống Kê Thời Gian
Tổng thời gian mở cửa (Giây)
150
100
50
0
23:37:00
23:41:00
23:48:00
Thống Kê Số Lượt (Đóng/Mở)
Số lượt đóng/mở
6
0
Lần
100
Điều Khiển
Chế độ bảo trì (Manual)
TRẠNG THÁI: ĐANG BẢO TRÌ


WOKWi Simulator
1.Đặng Văn Long
Community License
03:13.701
98%
PIR Motion Sensor
Simulate motion
ESP32
PROBLEMS
OUTPUT
TERMINAL
DEBUG CONSOLE
PORTS
Nhận lệnh từ MQTT: MANUAL_ON
Chế độ: THỦ CÔNG (Mở cửa tự do, vô hiệu hóa PIR)
powershell
Build Task


HỆ THỐNG QUẢN LÝ CỬA TỰ ĐỘNG
Thống Kê Thời Gian
Tổng thời gian mở cửa (Giây)
150
100
50
0
23:37:00
23:42:00
23:49:00
Thống Kê Số Lượt (Đóng/Mở)
Số lượt đóng/mở
0
Lần
0
100
Điều Khiển
Chế độ bảo trì (Manual)
TRẠNG THÁI: BÌNH THƯỜNG


WOKWI Simulator
1.Đặng Văn Long
Community License
04:33.587
92%
PIR Motion Sensor
Simulate motion
ESP32
PROBLEMS
OUTPUT
TERMINAL
DEBUG CONSOLE
PORTS
Data sent: {"event":"manual_mode","total_time_day":49,"cycle_count":5,"alert":"Manual door open","maintenance":"REQUIRED"}
Nhận lệnh từ MQTT: AUTO
Chế độ: TỰ ĐỘNG (Quay lại dùng cảm biến PIR)
powershell
Build Task
Wokwi Termin...

5. Kết luận hướng phát triển

5.1. Các công việc đã thực hiện được

Tính đến thời điểm hiện tại, hệ thống đã hoàn thiện được một nguyên mẫu (prototype) IoT hoạt động ổn định với các tính năng cốt lõi sau:

- Thiết kế phần cứng cơ bản: Lắp ráp và lập trình thành công mạch ESP32 kết nối với cảm biến chuyển động PIR, LED (mô phỏng cửa) và Còi (Buzzer).
- Logic điều khiển cục bộ: Hệ thống tự động nhận diện người để "mở cửa" (bật LED) và phát âm thanh phản hồi (bíp ngắn). Tích hợp thành công logic đếm thời gian thực tế để phát cảnh báo tại chỗ (hú còi) nếu cửa bị mở hoặc kẹt quá 10 phút.
- Truyền thông IoT & Đám mây: Thiết lập thành công giao thức MQTT qua HiveMQ Cloud để truyền nhận dữ liệu hai chiều. Tối ưu hóa việc gửi dữ liệu bằng định dạng JSON (gói gọn thời gian mở, số lượt, trạng thái cảnh báo và bảo trì).
- Xây dựng hệ thống giám sát (Dashboard): Triển khai thành công Node-RED để nhận dữ liệu, trực quan hóa qua biểu đồ (Chart), thước đo (Gauge) và các Popup thông báo trên giao diện Web.
- Hệ thống cảnh báo từ xa & Điều khiển: Tích hợp thành công tính năng tự động gửi cảnh báo qua Email khi có sự cố (quá giờ, cần bảo trì) và tính năng điều khiển chéo (Manual Override) cho ph***ép người dùng tắt cảm biến, ép mở cửa bằng nút gạt trên Dashboard.***

5.2. Các công việc chưa thực hiện được

Mặc dù hệ thống đã đáp ứng tốt các yêu cầu về mặt logic và truyền thông, nhưng vẫn còn một số hạn chế ở phiên bản nguyên mẫu này:

- Chưa có cơ cấu chấp hành thực tế: Hiện tại việc "mở cửa" mới chỉ dừng lại ở mức mô phỏng bằng đèn LED. Chưa kết nối với các động cơ kéo cửa (Servo, Motor DC) hoặc khóa điện từ (Solenoid Lock).
- Thiếu cơ sở dữ liệu lưu trữ dài hạn: Dữ liệu hiện đang được hiển thị trực tiếp trên Node-RED. Nếu Server Node-RED hoặc ESP32 khởi động lại, các số liệu lịch sử (biểu đồ các ngày trước) có thể bị mất do chưa được lưu vào Database (như MySQL hay InfluxDB).
- Chưa xử lý triệt để vấn đề mất điện lưới: Chưa được triển khai đồng bộ lên phần cứng vật lý.
- Hạn chế của cảm biến PIR: Cảm biến PIR thỉnh thoảng có thể bị nhiễu do nhiệt độ môi trường, ánh sáng mặt trời hoặc vật nuôi, dẫn đến việc báo "mở cửa" giả (False positive).

5.3. Hướng phát triển của đề tài

Để nâng cấp dự án này thành một sản phẩm thương mại hoặc một hệ thống nhà thông minh (Smart Home) hoàn thiện, đề tài có thể phát triển theo các hướng sau:

- Tích hợp phần cứng thực tế: Thay thế đèn LED bằng mạch Relay để điều khiển trực tiếp hệ thống cửa cuốn, cửa trượt tự động hoặc khóa điện từ.
- Đa dạng hóa phương thức xác thực (Bảo mật nhiều lớp): Kết hợp thêm Module đọc thẻ từ , cảm biến vân tay hoặc mật khẩu bàn phím. Cảm biến PIR lúc này sẽ chỉ đóng vai trò "đánh thức" hệ thống hoặc nhận diện người đi từ trong nhà ra.

- Tích hợp Camera an ninh (ESP32-CAM): Khi PIR phát hiện người và cửa mở, hệ thống sẽ tự động chụp một bức ảnh và gửi về Telegram hoặc Email cho chủ nhà để biết chính xác ai đang vào nhà.
- Xây dựng cơ sở dữ liệu & Ứng dụng di động: Lưu trữ toàn bộ lịch sử đóng/mở cửa vào Database để trích xuất báo cáo. Xây dựng một ứng dụng trên điện thoại di động (sử dụng Flutter hoặc Blynk) thay vì chỉ dùng Node-RED trên máy tính, giúp chủ nhà giám sát mọi lúc mọi nơi bằng một chạm.
- Tích hợp Cảnh báo cháy nổ: Thêm cảm biến khí gas (MQ-2) hoặc cảm biến lửa. Khi có cháy, hệ thống sẽ tự động ép mở cửa (bất kể trạng thái khóa) để người trong nhà thoát hiểm an toàn.

PHỤ LỤC

Phụ lục A:(Mã nguồn chương trình Arduino)

https://github.com/LongPrince/IoT_CamBienChuyenDong/blob/master/src/main.cpp

Phụ lục B:(Mã nguồn của Web-Node-Red(Json)

https://github.com/LongPrince/IoT_CamBienChuyenDong/blob/config-file/Nhom6.json

Phụ lục C:(Các nội dung khác Nếu có)

Mã nguồn của dự án:

https://github.com/LongPrince/IoT_CamBienChuyenDong.git

https://canva.link/v2qipxdy0rv8vp8